温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3101 W03 Foundations of Business Analytics

Pandas II

Dawei Chen

Week 3-2, 18 Sep 2025

Getting Help in Jupyter

- Quick Help
  - ?object or object? → show help/docstring
  - ??object or object?? → show more detail (e.g., source code, if available)
  - help(object) → open help in a pager window
- Tab Completion
  - object_name.<Tab> → list available methods and attributes
- General Help
  - Use the **Help** menu in Jupyter Notebook
  - Explore documentation (e.g., pandas documentation)

Examples & Demonstration

Calling Functions in Python

- Basic Form: `function_name(parameters)`
  - Built-in / defined earlier: `len()`
  - From imported modules: `pd.DataFrame()`, `pd.read_csv()`
  - Object's method: `data.head()`
- Parameters (参数)
  - Positional (必须参数): given in order → `round(3.14159, 2)`
  - Keyword (关键字参数): specify name → `print("hi", end="!")`
  - Default (默认参数): if not provided, default is used → `print("hi")`
  - Variable length (不定长参数): `*args`, `**kwargs`
- Tips
  - Check help: `function_name?`
  - Use Tab completion to explore methods
  - Remember: different functions accept different parameters → check the docstring
  - Pass variables instead of complex values for clarity
  - Separate parameters with commas `,`

Examples & Demonstration

Sequence of Running Code Matter

- Example

```
In [1]:  a = 5
In [2]:  b = a + 3
In [3]:  c = b + 5
```

  - Change a = 10, but only re-run the last line → c is still **13**
  - Re-run first and the third lines only → c is still **13** (because b not updated)

- Lesson
  - Order of running cells matters in Python/Jupyter Notebook
  - Variables values depend on the sequence of execution
  - Always check their current values before using them
- Tips
  - If results look wrong, check which cells have been run
  - In Jupyter Notebook: restart the kernel and re-run all cells for a clean state

Pandas – Essential Functionality

Label →		0	1	2	3	
		A	**B**	**C**	**D**	**Column**
0	**a**	1	2	3	4	
1	**b**	5	6	7	8	
2	**c**	9	10	11	12	
3	**d**	13	14	15	16	

Index

Many functions can achieve the same result. You don't need to remember them all—learn one method that works and use it well.

- Reindexing & Dropping
  - Rename, reorder, add, drop, sort, rank rows or columns
- Selecting & Filtering
  - Select and extract subsets based on conditions (e.g., values in a column)
- Arithmetic & Alignment
  - Arithmetic between Series or DataFrames
  - Automatic alignment on row & column labels
- Function Application & Mapping
  - Apply functions elementwise or along rows/columns or mapping function
  - Aggregate and summarize data
- Combination
  - Merge, join, concatenate two DataFrames vertically or horizontally

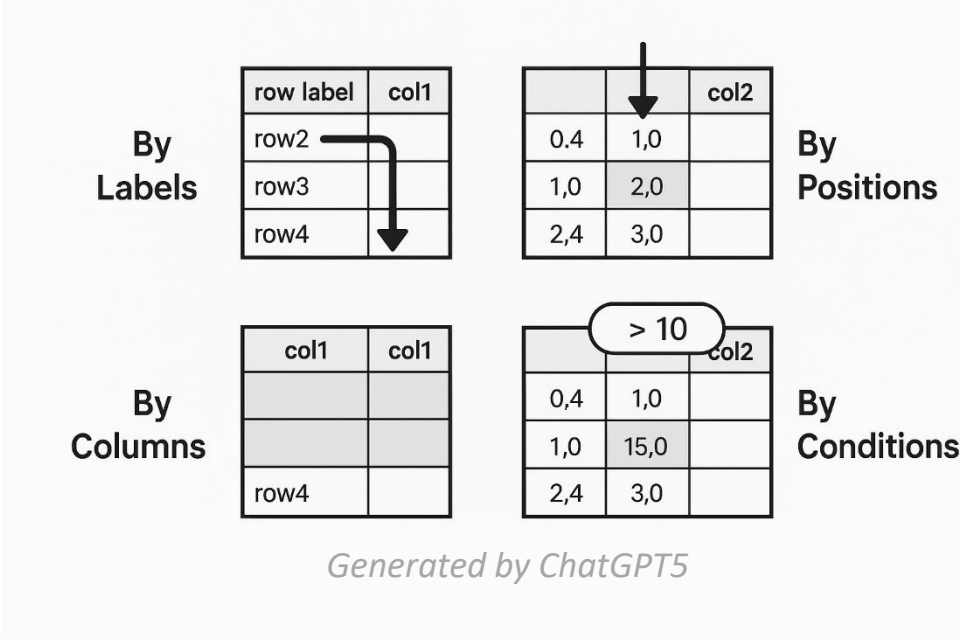
Pandas – Reindexing

- Rename & Reorder
  - *df.rename(columns={…}, index={…})* rename rows/columns
- Add Columns
  - *df[“new_col”] = values*
- Drop Rows/Columns
  - *df.drop[“col_name”, axis=1]* drop a column
  - *df.drop([“row1”, “row2”], axis=0)* drop rows
- Note
  - “new_col” or ‘new_col ’ → string (字符串)
  - new_col → variable (变量)

Pandas – Selecting & Filtering

df.loc() does not exit

- By Labels
  - *df.loc["row_label", "col_label"]* select rows/columns by names
- By Positions
  - *df.iloc[2, 0]* select rows/columns by integer index
- By Columns
  - Single column: *df["col"]*
  - Multiple columns: *df[["col1", "col2"]]*
- By Conditions
  - Boolean filtering: *df[df["col"] > 10]*
  - Combine conditions with & (and), | (or)
- By Slicing
  - Row slices: *df[2:5]*
  - With labels: *df.loc["row1":"row5"]*



Pandas – Arithmetic & Alignment

- Elementwise Operations
  - Add, subtract, multiply, divide directly on columns/rows
  - Example: *df["col1"] + df["col2"]*
- Automatic Alignment
  - Operations align on row/column labels
  - Missing values → result is *NaN*
- With Scalars
  - Apply arithmetic to all values
  - Example: *df["col1"] * 100*
- Function Application
  - Apply NumPy functions: *np.sqrt(df)*
  - Apply DataFrame methods: *df.mean(), df.sum()*
- Creating New Variables
  - Combine columns to form new variables
  - Example: *df["total"] = df["sales"] * df["price"]*

Don't just memorize – get familiar through practice.

Mini Task – Practicing DataFrame Basics

- Create a DataFrame with the Following Columns
  - “name”: [‘zhangsan’, ‘lisi’, ‘wangwu’, ‘zhaoliu’]
  - “age”: [20, 19, 21, 20]
  - “score”: [78, 89, 55, 61]
- Try the Following Operations
  - Rename the column “score” to “final_score”
  - Drop the “age” column
  - Display only “name” column
  - Display “name” and “final_score” columns together
  - Show only students with “final_score” is greater than 70
  - Add a new column: “bonus_score” whose values are: final_score + 5
  - Add a new column: “passed” (True if final_score >= 60, False otherwise)

When you work with a DataFrame, always ask:

- *Does this operation change the original DataFrame?*
- *Does it return a new object (e.g., a Series, a DataFrame, or an Index)?*
- *Does it change the value of existing variables?*

= assignment operator

Summary

- Additional Python Basics
- DataFrame Basic Operations
- Key Mindset
  - Practice is the best way to learn
  - Always ask: *Does this operation change the DataFrame or return something new?*
  - Don’t try to memorize everything – get comfortable through use
- Useful Resource
  - Pandas tutorial from W3Schools
- Individual Assignment 1
  - Due: **26 Sep 2025, 23:59**